NWT Module 5 Exam Answers

Client Side Scripting with JavaScript and DOM

Two-mark and five-mark answers with examples, comparison tables, and document trees. Question numbers follow the separate marks sections of the supplied question bank.

2 Marks | Q9. Analyze the uses of the Document Object Model (DOM) in developing dynamic web pages.

1. The DOM represents a web page as a tree of objects that JavaScript can access and modify.

2. It allows developers to change text, images, and other content without reloading the entire page.

3. It enables JavaScript to modify CSS styles, such as changing an element's colour or hiding a section.

4. It allows elements to be added or removed dynamically, such as adding a new item to a shopping cart.

5. It supports user interaction through event handling, such as displaying a message when a button is clicked.

2 Marks | Q10. Examine the structure of the Document Object Model (DOM) and infer its importance in web programming.

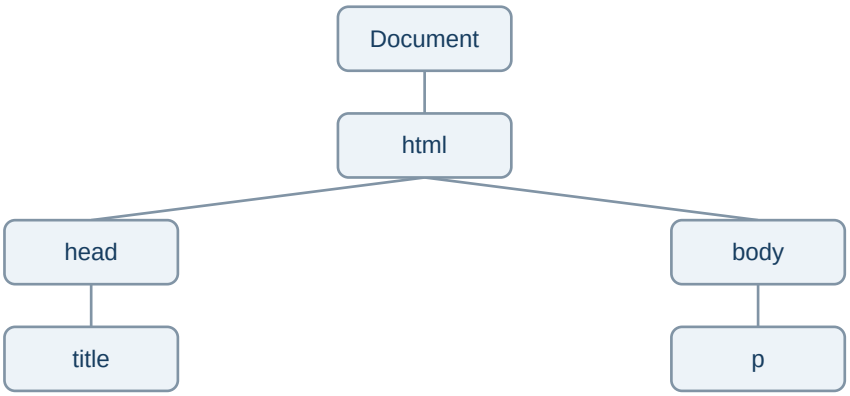
1. The DOM represents an HTML document as a hierarchical tree containing different types of nodes.

2. The document node represents the entire document, while the <html> element is its root element.

3. HTML elements, such as <head>, <body>, and <p>, form element nodes, while their text forms text nodes.

4. Nodes have parent, child, and sibling relationships that allow JavaScript to navigate the document.

5. This structure helps developers access and modify specific elements to create interactive web pages.



The title and paragraph can each contain a text node, such as "My Page" and "Hello".

2 Marks | Q11. Discuss the history of the Document Object Model (DOM).

1. The DOM originated in the mid-1990s when JavaScript and browser scripting created a need to access web page elements.

2. Early browser-specific interfaces developed by Netscape and Microsoft became informally known as DOM Level 0.

3. The W3C introduced DOM Level 1 in 1998 to standardise access to HTML and XML document structures.

4. DOM Level 2 introduced standardised event handling and additional support for styles and document traversal, while Level 3 expanded its capabilities.

5. Modern DOM development follows a living standard and provides features such as querySelector() and addEventListener().

2 Marks | Q12. Compare the different levels of event handling and select the appropriate event-handling mechanism for a web application.

Basis	DOM Level 0	DOM Level 2
Registration	A handler is assigned through an HTML attribute or an element property, such as onclick.	A handler is registered using addEventListener().
Multiple handlers	An event-handler property holds one handler, so another assignment replaces it.	Multiple listeners can be registered for the same event on an element.
Event phases	Property-based handlers do not provide an option to register for the capture phase.	Listeners can be registered for the capture phase or the normal non-capture phase.
Removal	A property-based handler can be removed by setting the property to null.	A listener can be removed using removeEventListener() with a matching listener and capture setting.
Suitability	This approach is suitable for simple pages with basic event handling.	This approach is preferred for complex applications because it supports multiple listeners and better separation of code.

2 Marks | Q13. Discuss real-life examples of using DOM in web pages.

1. A shopping website uses the DOM to update the cart item count when a user adds a product.
2. A registration form uses the DOM to display validation errors beside incorrectly filled fields.
3. A messaging application uses the DOM to insert newly received messages into the conversation.
4. A website uses the DOM to open or close a navigation menu when its menu button is clicked.
5. A search interface uses the DOM to display suggestions as the user types into the search box.

2 Marks | Q14. Compare the DOM with the HTML source code and infer their roles in web page rendering.

Basis	HTML source code	DOM
Representation	HTML source code is the textual markup used to describe a web page.	The DOM is the object-based tree created when the browser parses that markup.
Structure	HTML expresses structure using tags, attributes, and text.	The DOM expresses structure using nodes and their relationships.
Modification	The original HTML source generally remains unchanged when JavaScript updates the page.	JavaScript can modify the DOM while the page is running.
Rendering	HTML source provides the initial instructions from which the browser builds the document.	The DOM provides the document structure used alongside styles to render the page.
Example	The source may initially contain Hello .	JavaScript can change that paragraph's DOM text to "Welcome", updating the displayed page.

2 Marks | Q15. Apply the process of creating an XML document for a given data representation.

1. Identify the data to represent and choose meaningful element names, such as student, name, and rollNo.

2. Optionally begin the document with an XML declaration specifying its version and character encoding.

3. Create exactly one root element that contains all the other elements.

4. Ensure that tags are properly nested and closed, names use consistent letter case, and attribute values are quoted.

5. Save the document with a .xml extension and check that it is well-formed.

```
<?xml version="1.0" encoding="UTF-8"?>
<student>
    <name>Aayush Gupta</name>
    <rollNo>35</rollNo>
    <division>TT AI&amp;amp;DS A</division>
</student>
```

2 Marks | Q16. Examine the DOM tree and analyze the event-handling process in web applications.

1. The DOM tree organises web page elements into nodes connected through parent-child relationships.

2. An event occurs when a user performs an action, such as clicking a button or typing into an input field.

3. A developer registers an event listener on a DOM element to respond to a particular event.

4. The event travels through the capture phase to its target and, if it bubbles, then moves through its ancestors.

5. The registered handler executes JavaScript that can update the page's content, structure, or appearance.

```
<button id="btn">Click Me</button>
<p id="message">Waiting...</p>
<script>
  document.getElementById("btn").addEventListener("click", function () {
    document.getElementById("message").textContent = "Button clicked!";
  });
</script>
```

Clicking the button runs its listener and changes the paragraph to "Button clicked!".

5 Marks | Q8. Illustrate the syntax, variables, and data types used in JavaScript.

1. **Syntax:** JavaScript is a case-sensitive scripting language used to add behaviour to web pages. Statements contain instructions, and curly braces group statements into blocks such as functions and conditions.

2. **Variables:** A variable is a named storage location used to hold a value. JavaScript uses var, let, and const to declare variables, as shown in let age = 20;.

3. **Using var:** A variable declared with var has function scope when declared inside a function. It can be redeclared and reassigned, although modern JavaScript generally prefers let and const.

4. **Using let:** A variable declared with let has block scope, so it is accessible within its declaring block. Its value can be reassigned, but it cannot be redeclared in the same scope.

5. **Using const:** A variable declared with const has block scope and must be initialised during declaration. Its binding cannot be reassigned, although properties of an object stored in it can still be modified.

6. **Data types:** JavaScript supports strings, numbers, booleans, undefined, null, symbols, big integers, and objects. Arrays and functions are specialised objects used for collections and executable behaviour.

7. **Dynamic typing:** A variable's type is determined by its current value. A variable can initially store a number and later store a string without a separate type declaration.

8. **Operators and comments:** Operators perform operations such as arithmetic, assignment, and comparison, using +, =, and ===. Single-line comments begin with //, while multiline comments use /* ... */.

Feature	var	let	const
Scope	It has function scope inside a function.	It has block scope.	It has block scope.
Redeclaration	It allows same-scope redeclaration.	It prohibits same-scope redeclaration.	It prohibits same-scope redeclaration.
Reassignment	Its value can be reassigned.	Its value can be reassigned.	Its binding cannot be reassigned.
Initialisation	An initial value is optional.	An initial value is optional.	An initial value is required.

```
const college = "TCET";
let marks = 80;
let passed = true;
marks = marks + 5;
if (passed) {
    console.log(college + ": " + marks);  // TCET: 85
}
```

5 Marks | Q9. Analyze XML syntax and structure in detail.

1. **Meaning and purpose:** XML stands for Extensible Markup Language and represents, stores, and exchanges structured data. Developers create meaningful tags that describe the data rather than its visual appearance.

2. **Declaration:** An XML document may begin with a declaration specifying the XML version and character encoding. When present, it must appear at the beginning of the document.

3. **Single root:** Every XML document must contain exactly one root element enclosing all other elements. For example, students can be the root element containing several student elements.

4. **Elements:** Elements generally use an opening tag, content, and a matching closing tag, such as <name>Aayush</name>. Empty elements can use the self-closing form, such as <address />.

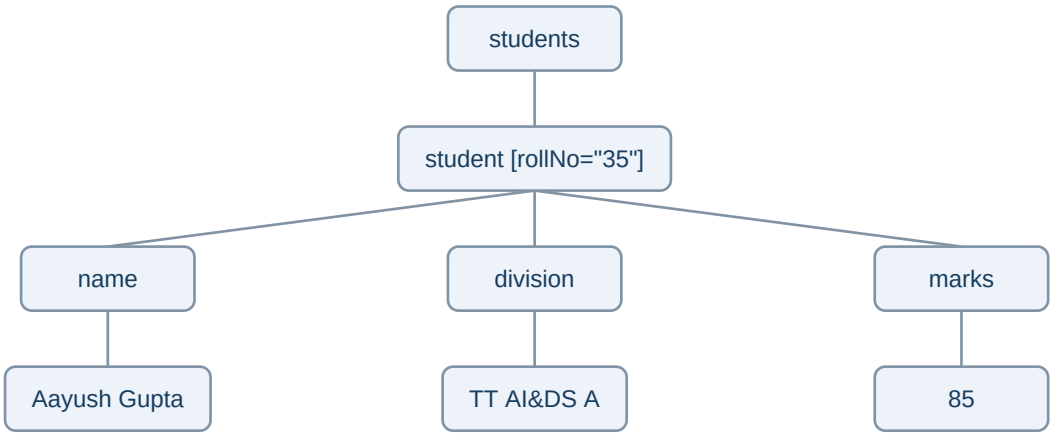
5. **Nesting and case:** Inner elements must close before their enclosing elements close. XML is case-sensitive, so <name> and <Name> are different element names.

6. **Attributes:** Attributes provide additional information inside an opening tag. Their values must be quoted, as shown in <student rollNo="35">.

7. **Special characters and comments:** Reserved characters must be escaped in ordinary text, using & for an ampersand and < for a less-than sign. Comments use <!-- Comment --> and do not form part of the data content.

8. **Well-formedness and validity:** A well-formed document follows XML syntax rules, including proper nesting and a single root. A valid document also conforms to an applicable definition such as a DTD; XML Schema can also check document structure and data constraints.

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
    <student rollNo="35">
        <name>Aayush Gupta</name>
        <division>TT AI&amp;DS A</division>
        <marks>85</marks>
    </student>
</students>
```



5 Marks | Q10. Explain the seven data types in JavaScript.

For this question, the seven-type classification is Number, String, Boolean, Undefined, Null, Object, and Symbol. Modern JavaScript additionally includes BigInt, so this is not the complete modern classification.

1. **Number:** This type represents integers and floating-point values, such as 35 and 85.5. It supports arithmetic and includes special values such as NaN and Infinity.

2. **String:** This type represents text enclosed in single quotes, double quotes, or backticks. Backticks support template literals, which can insert expressions into text.

3. **Boolean:** This type has only true and false, representing logical outcomes. It is commonly used in conditions, such as checking whether a student has passed.

4. **Undefined:** This type indicates that a value has not been assigned. After let result;, result is undefined until another value is assigned.

5. **Null:** This type represents the intentional absence of a value. For example, let selectedStudent = null; indicates that no student is currently selected.

6. **Object:** This type stores collections of properties, associating keys with values. Arrays and functions are also objects, representing collections and executable operations.

7. **Symbol:** This type represents a unique primitive value that can serve as an object property key. Two symbols created with the same description are different, helping prevent property-name conflicts.

Type	Example	Explanation
Number	let marks = 85.5;	It stores a numeric value.
String	let name = "Aayush";	It stores textual data.
Boolean	let passed = true;	It stores a logical value.
Undefined	let result;	It has no assigned value.
Null	let selected = null;	It intentionally represents no selected value.
Object	let student = {name: "Aayush", marks: 85};	It stores related values as properties.
Symbol	let id = Symbol("id");	It stores a unique symbol value.

```
let first = Symbol("id");
let second = Symbol("id");
console.log(first === second);  // false
```

5 Marks | Q11. Illustrate client-side scripting using JavaScript.

1. **Meaning:** Client-side scripting executes scripts inside the user's web browser. JavaScript makes web pages interactive and responsive.

2. **Embedding:** JavaScript can appear inside a script element or load from an external .js file. External files organise code and allow scripts to be reused across pages.

3. **DOM manipulation:** JavaScript accesses the DOM to modify content, structure, and styles. It can change a heading, display a notification, or add a list item.

4. **Event handling:** JavaScript responds to clicks, keyboard input, and form submissions. An event listener connects an event with a function that performs the required action.

5. **Form validation:** Client-side scripts check required fields and input formats, providing immediate feedback. The server must also validate submitted data because browser checks can be bypassed.

6. **Asynchronous communication:** APIs such as fetch() allow communication with a server without a full-page reload. Applications can update search results, messages, and other content in the background.

7. **Advantages and limitations:** Local processing provides quick responses and can reduce unnecessary server requests. However, behaviour depends on browser support, and downloaded code is accessible to the user.

```
<input id="name" type="text" placeholder="Enter your name">
<button id="btn">Submit</button>
<p id="message"></p>
<script>
  document.getElementById("btn").addEventListener("click", function () {
    const name = document.getElementById("name").value.trim();
    const message = document.getElementById("message");
    if (name === "") {
      message.textContent = "Please enter your name.";
      message.style.color = "red";
    } else {
      message.textContent = "Welcome, " + name + "!";
      message.style.color = "green";
    }
  });
</script>
```

Explanation: Clicking Submit executes the script in the browser. An empty input produces a red validation message, while a valid name produces a green welcome message without reloading the page.

5 Marks | Q12. Explain the main functions of the Document Object Model (DOM).

1. **Representing the document:** The DOM represents HTML or XML as a hierarchical tree of nodes. It provides a standard way for programs to interact with the document.

2. **Accessing elements:** Methods such as getElementById() and querySelector() locate elements. For example, document.getElementById("title") selects the element whose ID is title.

3. **Modifying content:** Scripts change text through textContent or HTML content through innerHTML. Pages can dynamically display messages, scores, and other information.

4. **Modifying attributes:** Methods such as getAttribute() and setAttribute() read and change attributes. Changing an image's src attribute displays a different image.

5. **Changing styles and classes:** The style property and classList modify an element's appearance. These operations can highlight errors, activate dark mode, or show and hide sections.

6. **Creating and removing elements:** createElement(), appendChild(), and remove() alter the document structure. A shopping application can add or remove products from its displayed cart.

7. **Handling events:** DOM event APIs respond to clicks, typing, and other actions through registered listeners. A button's listener can open a menu or update a paragraph.

8. **Navigating the tree:** parentElement, children, and nextElementSibling navigate between related elements. This is useful when processing lists, forms, and nested sections.

```
<h2 id="title">Shopping List</h2>
<ul id="list"></ul>
<button id="addBtn">Add Item</button>
<script>
  document.getElementById("addBtn").addEventListener("click", function () {
    const item = document.createElement("li");
    item.textContent = "Notebook";
    document.getElementById("list").appendChild(item);
    document.getElementById("title").style.color = "blue";
  });
</script>
```

Explanation: The script selects elements, handles a click, creates a list item, inserts it into the DOM, and changes the heading's colour.

References: Supplied NWT question bank and Module 5 notes (Unit 5 and NWT UNIT 5).